

D&DSL

Uma DSL para rolagens de dados de RPG, escrita em Python com Lark

Joel Júnior

O que é a D&DSL

Uma linguagem pequena, feita para um único problema: descrever rolagens de dados de RPG.

O problema

Rolagens de RPG misturam vários dados, descartes, rerolls e contas. Em Python isso vira código longo e fácil de errar.

A solução

Uma notação curta e legível: `4d6kb3 + 2` rola quatro d6, mantém os três maiores e soma 2.

O resultado

Um parser em Lark valida a expressão e um interpretador rola os dados e mostra o detalhe de cada rolagem.

A mesma rolagem, duas linguagens

EM PYTHON

```
sum(sorted(
    random.randint(1, 6)
    for _ in range(4)
)[-3:]) + 2
```

EM D&DSL

4d6kb3 + 2

Vocabulário do domínio: dados, faces, manter, rerolar.

Menos código e menos chance de erro de lógica.

Entrada inválida vira erro de sintaxe, não bug silencioso.

Por que o domínio é adequado

Vocabulário pequeno e fechado

Poucos verbos: rolar, manter, rerolar, explodir e somar.

Notação já conhecida

A forma XdY é uma convenção comum entre jogadores de RPG.

Semântica bem definida

Cada modificador tem uma regra clara e um resultado numérico.

Entradas curtas e reutilizáveis

Uma expressão vale em qualquer plataforma com o interpretador.

Onde a DSL pode ser usada

Bots de chat



Um comando de rolagem para servidores de RPG e grupos de mensagens.

Mesas virtuais



Rolagens em fichas, macros e automações de sistemas de RPG.

Ensino de linguagens formais



Um exemplo pequeno que reúne gramática, árvore e interpretador.

Programas válidos

1d20	um dado de 20 faces
2d6 + 3	soma de dois d6 com bônus fixo de 3
4d6kb3	rola 4d6 e mantém os 3 maiores (keep biggest)
2d20ks1	rola 2d20 e mantém o menor (keep smallest)
4d6 rr<3 kb3	rerola dados menores que 3, depois mantém os 3 maiores
1d6 ex	dado no valor máximo rola de novo e soma
(2d6 + 3) * 2	parênteses mudam a precedência
1d10+5+1d6*2	várias rolagens na mesma expressão

Entradas rejeitadas

4d6 rr3	o rr precisa de < ou > antes do número
2d6 3d6	duas expressões sem operador entre elas
(2d6 + 3	parêntese aberto e nunca fechado
4d6 kh3	kh não existe; os modificadores são kb e ks
d6	falta a quantidade de dados
2d6 +	operador sem segundo operando

Todas geram erro de sintaxe do Lark, com linha e coluna.

Tokens da linguagem

TOKEN	COMO É RECONHECIDO	EXEMPLO
NUMBER	inteiro sem sinal (common.INT)	1, 20, 100
COMP	comparador do reroll	< >
"d"	separa quantidade de faces	2d6
"ex" "kb" "ks" "rr"	palavras-chave dos modificadores	4d6 kb3
"+" "-" "*"	operadores aritméticos	2d6 + 3
"(" ")"	agrupamento	(2d6 + 3) * 2
espaços	ignorados (%ignore WS)	4d6 kb3 = 4d6kb3

Gramática em EBNF

```
start      ::=  expr
expr       ::=  expr "+" term | expr "-" term | term
term       ::=  term "*" factor | factor
factor     ::=  roll | NUMBER | "(" expr ")"
roll       ::=  NUMBER "d" NUMBER modifier*
modifier   ::=  "ex" | "kb" NUMBER | "ks" NUMBER | "rr" COMP NUMBER
COMP       ::=  "<" | ">"
NUMBER     ::=  ("0" .. "9")+
```

* zero ou mais • + um ou mais • texto entre aspas é terminal • espaços em branco são ignorados

Testando no Lark

```
from dice_dsl_4 import parser, run_roll
parser.parse("4d6 rr<3 kb3")    # só valida
run_roll("4d6 rr<3 kb3")       # valida e executa
```

4d6 → dados: [2, 5, 1, 3] → reroll <3: [1, 5, 4, 3] → kb3: [5, 4, 3] → total: 12
⇒ resultado final: 12

Árvore de análise de 4d6kb3 + 2

```
start
  add_expr
    expr
      term
        factor
          roll
            4
            6
            keep_biggest 3
      term
        factor 2
```

A estrutura faz sentido?

Sim. A raiz é `add_expr`: a soma é a última operação a ser calculada.

À esquerda está o dado: `roll` com 4 (quantidade), 6 (faces) e o modificador `keep_biggest 3`.

Os modificadores são filhos de `roll`, na ordem em que foram escritos. A ordem de execução fica com o interpretador.

À direita, o 2 é um `factor` solto: o bônus da soma.

O + e o d não aparecem: o Lark descarta literais anônimos, e o alias `add_expr` guarda a operação.

Derivação de 4d6kb3 + 2 (1/2)

Cada passo expande o não terminal mais à esquerda. Em itálico âmbar: ainda falta expandir.

1	<i>start</i>	início
2	<i>expr</i>	start → expr
3	<i>expr + term</i>	expr → expr "+" term
4	<i>term + term</i>	expr → term
5	<i>factor + term</i>	term → factor
6	<i>roll + term</i>	factor → roll
7	<i>NUMBER d NUMBER modifier* + term</i>	roll → NUMBER "d" NUMBER modifier*
8	<i>NUMBER d NUMBER modifier + term</i>	modifier* → modifier (uma repetição)

Derivação de 4d6kb3 + 2 (2/2)

9	<i>NUMBER</i> d <i>NUMBER</i> kb <i>NUMBER</i> + <i>term</i>	modifier → "kb" NUMBER
10	4 d <i>NUMBER</i> kb <i>NUMBER</i> + <i>term</i>	NUMBER → 4
11	4 d 6 kb <i>NUMBER</i> + <i>term</i>	NUMBER → 6
12	4 d 6 kb 3 + <i>term</i>	NUMBER → 3
13	4 d 6 kb 3 + <i>factor</i>	term → factor
14	4 d 6 kb 3 + <i>NUMBER</i>	factor → NUMBER
15	4 d 6 kb 3 + 2	NUMBER → 2

Resultado: 4d6kb3 + 2, igual à entrada.

1. Notação familiar e curta

Usar os idiomas que o público já conhece, com palavras-chave curtas.

XdY

quantidade e faces, como em
qualquer mesa de RPG

kb ks

keep biggest e keep smallest:
siglas curtas em inglês

rr

reroll de dados abaixo ou
acima de um limite

Brevidade proporcional ao tamanho da linguagem.

2. Precedência pela estrutura

expr soma e subtração

chama ↓

term multiplicação

chama ↓

factor dado, número ou (expr)

Regra prática

Quanto mais fundo na gramática, mais forte o operador prende.

$2d6 + 3 * 2$ vira $2d6 + (3 * 2)$: o $*$ fica em term, abaixo do $+$.

Os parênteses fecham o ciclo: factor volta para expr.

Existe precedência entre os modificadores?

Na gramática atual, `modifier*` aceita `ex`, `kb` e `rr` em qualquer ordem. Isso significa que não existe nenhuma precedência entre eles?

O que temos hoje

```
roll ::= NUMBER "d" NUMBER modifier*  
modifier ::= "ex" | "kb" NUMBER | "ks"  
NUMBER | "rr" COMP NUMBER
```

`modifier*` não impõe nenhuma ordem entre os modificadores.

O que teríamos com camadas

```
roll ::= kept kept ::= kept "kb" NUMBER  
| kept "ks" NUMBER | exploded exploded  
::= exploded "ex" | rerolled rerolled  
::= rerolled "rr" COMP NUMBER | base  
base ::= NUMBER "d" NUMBER
```

Cada regra só recebe a de baixo: a ordem de escrita passa a ser obrigatória, `rr` → `ex` → `kb/ks`.

3. Ordem dos modificadores

A sintaxe aceita os modificadores em qualquer ordem. O interpretador sempre executa nesta:



✓ 4d6 rr<3 kb3

escrito na ordem de execução

✓ 4d6 kb3 rr<3

fora de ordem: mesma execução e mesmo resultado

Quem escreve não precisa decorar a ordem, e nunca se mantém dados antes de rerolar. A alternativa seria uma gramática em camadas, que força a ordem na sintaxe e rejeita a segunda linha.

5. Regra inicial, aliases e reuso

`start: expr`

Um programa é uma expressão só. Com `expr+`, `2d6 3d6` seria aceito e a segunda expressão seria ignorada.

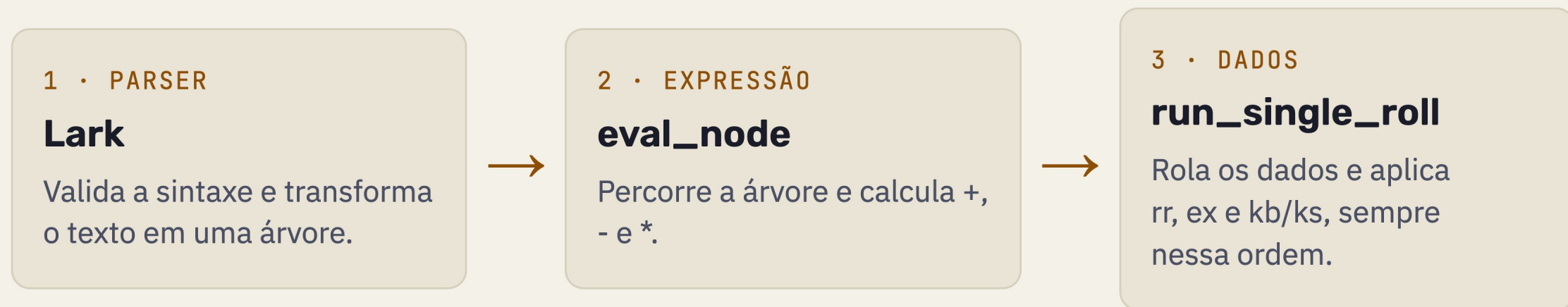
`→ add_expr`

O Lark descarta literais anônimos como `+` e `d`. O nome do alias guarda a operação na árvore.

`common.INT`

Reaproveita o terminal pronto. O `common.NUMBER` aceitaria decimais, que dados não têm.

Como o programa é executado



Um dado sozinho mostra só o total. Com mais de um dado ou alguma operação, o resultado de cada dado é mostrado.

Rodando a D&DSL

`4d6kb3 + 2`

`4d6` → dados: [4, 5, 3, 4] → `kb3`: [5, 4, 4] → total: 13

⇒ resultado final: 15

`1d10+5+1d6*2`

`1d10` → dados: [9] → total: 9

`1d6` → dados: [5] → total: 5

⇒ resultado final: 24

`4d6 rr<3 kb3`

`4d6` → dados: [2, 5, 1, 3] → reroll <3: [1, 5, 4, 3] → `kb3`: [5, 4, 3] → total: 12

⇒ resultado final: 12

`4d6 kb3 rr<3`

`4d6` → dados: [2, 5, 1, 3] → reroll <3: [1, 5, 4, 3] → `kb3`: [5, 4, 3] → total: 12

⇒ resultado final: 12

Capturas de execuções reais; sem semente fixa, o resultado muda a cada rodada.

D&DSL

Obrigado!

Perguntas?